

Final Documentation

Project Title: Budget Buddy

Problem Statement

Many people find it difficult to keep track of their spending because they rely on messy note-taking or forget to record their expenses consistently. This lack of organization leads to overspending, financial stress, and difficulty saving money. The real problem is the absence of a simple and organized system that allows users to monitor their daily expenses and remaining budgets in real-time. This project aims to solve that problem by creating a Budget Tracker program that automatically updates a person's remaining budget each time they record an expense. It helps users visualize where their money goes, make better financial decisions, and develop better budgeting habits.

Project Objective (SMART Framework)

- **Specific:** To create a Python program that helps users monitor and manage their weekly budget by tracking expenses under specific categories.
- **Measurable:** The program will display total budget balances after every recorded expense.
- **Achievable:** The project will be developed using basic Python programming concepts and completed by the end of the school year.
- **Relevant:** The project promotes financial awareness and responsible spending habits.
- **Time-bound:** The program will be fully developed and functional by the end of the 4th Quarter.
- **Overall Objective:** To design and implement a Python-based Budget Tracker that helps users monitor weekly spending, prevent overspending, and build better budgeting habits through categorized expense tracking and automatic budget updates.

Features

- **Budget Input** – Allows the user to enter their original weekly budget.
- **Expense Recording** – Allows the user to input daily expenses for specific categories.
- **Automatic Budget Update** – Deducts the spent amount from the weekly budget and recalculates the remaining balance.
- **Weekly Balance Calculation** – Updates and displays the user's remaining total weekly budget based on all category expenses.

- Data Display – Presents a summary showing each category, total spent, and remaining balance.
- Continuous Tracking – Lets the user enter new expenses throughout the week and see updated results.

Removed Features

- Category Creation – Lets the user create multiple expense categories such as food, transport, or entertainment.
 - Removed because it would make the program more complicated
- Category Budget Allocation – Enables the user to assign an initial weekly budget to each category.
 - Removed because it is inefficient

Added Features

- Reset button - Lets the user start a new week
- Exit button - Lets the user exit the program with all the data saved so that it is not lost

FILE / PROJECT STRUCTURE

Main File

budget_app.py

Everything is currently inside one file, but logically it is divided into sections like this:

1. DATA / STATE SECTION

Stores all global variables used across the app.

Variables:

- `subgroups` → expense categories
- `days` → week days
- `used_days` → tracks completed days
- `weekly_data` → stores daily total expenses
- `current_day` → active selected day
- `weekly_budget` → user budget
- `SAVE_FILE` → JSON save location
- `last_screen` → remembers last UI screen

2. SAVE / LOAD SYSTEM

Handles persistence (data saving even after closing app)

Functions:

- `save_to_file()`
 - Saves all app data into `budget_data.json`
- `load_from_file()`
 - Loads saved data when app starts

3. APP EXIT HANDLER

- `on_close()`
 - Saves data then closes the app safely

4. RESET SYSTEM

- `reset_week()`
 - Clears all progress
 - Resets budget, days, and data
 - Brings user back to start screen

5. SUMMARY SYSTEM

- `get_summary_text()`
 - Generates text summary of:
 - Budget
 - Expenses per day
 - Total spent
 - Remaining balance

6. START SCREEN LOGIC

- `start_app()`
 - Validates budget input
 - Converts input to float
 - Starts app and goes to day selection screen

7. DAY SELECTION SCREEN

- `day_selection_screen()`
 - Main navigation screen
 - Shows:
 - Summary panel (left)
 - Available days (right)

- Reset & Exit buttons

8. INPUT SCREEN (EXPENSE ENTRY)

- `input_screen(day)`
 - Lets user input expenses per category:
 - School
 - Food
 - Transportation
 - Personal Expense

9. PREVIEW SCREEN

- `preview_screen()`
 - Shows:
 - Category breakdown
 - Total spending
 - Validates inputs (no negative numbers)

10. SAVE DAILY DATA

- `save_data()`
 - Saves daily total into `weekly_data`
 - Marks day as used
 - Moves to review screen

11. REVIEW SCREEN

- `review_screen()`
 - Final confirmation of day's expenses
 - Shows full breakdown again
 - Allows navigation back

12. UI UTILITY

- `clear(frame)`
 - Deletes all widgets inside a frame
 - Used for switching screens

13. MAIN UI SETUP

- Creates Tkinter window:
 - `root`
 - `start_frame`

- `main_frame`
- Sets:
 - window size
 - title
 - close behavior

14. AUTO LOAD + RESTORE STATE

- `load_from_file()`
- Automatically restores:
 - last screen
 - budget
 - saved expenses

New Technologies/Tools

- **Tkinter**
 - Tkinter is a standard Python library used for creating graphical user interfaces (GUI), including windows, buttons, and input fields.
- **Python IDLE**
 - Python IDLE is the default, lightweight software tool bundled with Python that allows you to write, edit, run, and debug Python code.
- **VsCode**
 - VS Code is a code editor that supports multiple programming languages and extensions

Methodology

Technologies

- **Programiz**
 - Programiz is an online platform that provides tutorials, documentation, and an

interactive compiler for programming languages such as Python.

We used this technology because it is efficient, easily accessible and popular among coders.

- **Visual Studio Code**

- Visual Studio Code (VS Code) is a lightweight but powerful code editor that supports multiple programming languages and extensions.

- We used this as a backup compiler because it offers more features needed to make the code-making process more efficient.

- **Tkinter**

- Tkinter is a standard Python library used for creating graphical user interfaces (GUI), including windows, buttons, and input fields.

- We used this for transforming the code into a GUI.

- **Google**

- Google is a web search engine that provides access to a wide range of online resources and documentation.

- We used this for research on Python coding(eg. [Python.org](https://python.org) , [Code.org](https://code.org) , Codecademy)

- **YouTube**

- YouTube is a video-sharing platform that offers tutorials and educational content.

- We used this for GUI conversion tutorials.

- **Python IDLE**

- Python IDLE is the default, lightweight software tool bundled with Python that allows you to write, edit, run, and debug Python code. We used this for python extension.

Backend–Frontend Communication

The program does not have a separate backend and frontend. Instead, both are integrated within a single file.

- The **frontend** is implemented using Tkinter widgets such as buttons, labels, and input fields.
- The **backend** consists of Python functions that handle logic, calculations, and data storage.

Communication between the two occurs through **event-driven programming**, where user interactions (e.g., button clicks) trigger functions that process data and update the interface.

Key Design Decisions and Trade-offs

The program uses a step-by-step screen flow (start → day selection → input → preview → summary) to make it easier for users to navigate. This improves usability but makes the code longer and more complex.

It also uses global variables like `weekly_data` and `current_day` to share data between screens. This simplifies the program but can make debugging harder.

The app relies on basic Tkinter widgets to keep it lightweight and easy to run without extra tools. However, this limits visual design and modern UI features.

Input validation with try-except ensures users enter valid numbers, improving reliability but giving only basic error messages.

Ethical Considerations in Programming Choices

The program respects user privacy by keeping all data local and not storing or sending it anywhere. However, this means data is lost when the app closes.

It is somewhat accessible due to simple labels and buttons, but it lacks features like screen reader support or accessibility settings.

The app also encourages responsible budgeting by tracking expenses and warning users when they exceed their budget.

Finally, input validation helps prevent errors, ensuring users get accurate and safe results.

References

Python Software Foundation. (n.d.). *Python documentation*. <https://docs.python.org/3/>

Python Software Foundation. (n.d.). *The Python tutorial*. <https://docs.python.org/3/tutorial/>

Code.org. (n.d.). *Learn computer science*. <https://code.org/>

Codecademy. (n.d.). *Learn Python 3*. <https://www.codecademy.com/learn/learn-python-3>

Programiz. (n.d.). *Python programming tutorials*.
<https://www.programiz.com/python-programming>

freeCodeCamp.org. (2020, September 21). *Python Tkinter GUI tutorial for beginners* [Video]. YouTube. <https://www.youtube.com/watch?v=YXPYB4XeYLA>

Programming with Mosh. (2019, May 2). *Python full course for beginners* [Video]. YouTube. <https://www.youtube.com/watch?v=uQrJ0TkZlc>

NeuralNine. (2021, June 10). *Tkinter Beginner Course - Python GUI Development* [Video].
YouTube. <https://www.youtube.com/watch?v=ibf5cx221hk>